Objective: In this lab, we will work on counter, flipflops and see their characteristics in the De-Soc1 board. Furthermore, we will be able to have a basic understanding of counters and sufficient familiarity with the VHDL hardware description language to implement various types of latches and flip-flops.

Part 1: In this part We use the VHDL code of t-flipflop to design a 8 bit counter and see the results on 7-segment display as well as send the output to LEDR. In addition, I design a 4-bit t-flipflop to understand the basic difference and see the RTL view of this circuit. For implementing, 8-bit counter We need to use 7 logic elements. The screenshot of outputs and codes are given below:

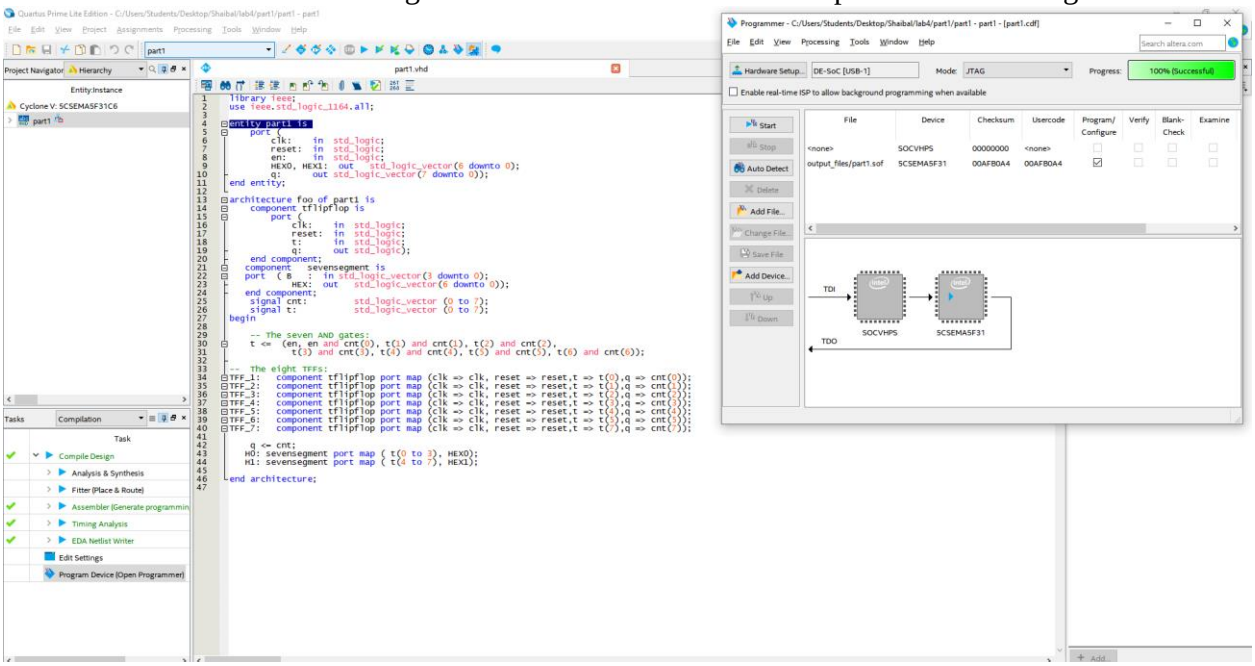


Figure: Code of part1

```
--------------------------------------------------------------------------------
-- Shaibal Saha
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity tflipflop is
    port (
        clk:    in  std_logic;
        reset:  in  std_logic;
        t:      in  std_logic;
        q:      out std_logic
    );
end entity tflipflop;

architecture behavioral of tflipflop is
    signal q_reg:   std_logic;
    signal q_next:  std_logic;
begin

T_FF:
    process (clk, reset)
    begin
        if reset = '1' then
            q_reg <= '0';
        elsif rising_edge(clk)  then
            q_reg <= q_next;
        end if;
    end process;

INPUT_MUX:
    q_next <=     q_reg  when t = '0' else
                not q_reg;
OUTPUT:
    q  <= q_reg;

end architecture;
```

Figure: Code of part1

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;


entity   sevensegment is
port  ( B    :  in std_logic_vector(3 downto 0);
        HEX:  out   std_logic_vector(6 downto 0));
end sevensegment;

architecture behavior of sevensegment is
begin
    with B select
    HEX <="1000000" when "0000", -- 0
          "1111001" when "0001", -- 1
          "0100100" when "0010", -- 2
          "0110000" when "0011", -- 3
          "0011001" when "0100", -- 4
          "0010010" when "0101", -- 5
          "0000010" when "0110", -- 6
          "1111000" when "0111", -- 7
          "0000000" when "1000", -- 8
          "0010000" when "1001", -- 9
          "0001000" when "1010", -- A
          "0000011" when "1011", -- b
          "1000110" when "1100", -- C
          "0100001" when "1101", -- d
          "0000110" when "1110", -- E
          "0001110" when "1111", -- F
          "1111111" when others; -- turn off all LEDs
end behavior;
```

Figure: Code of part1

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔		out HEX0[0]	Location	PIN_AE26	Yes			
2	✔		out HEX0[1]	Location	PIN_AE27	Yes			
3	✔		out HEX0[2]	Location	PIN_AE28	Yes			
4	✔		out HEX0[3]	Location	PIN_AG27	Yes			
5	✔		out HEX0[4]	Location	PIN_AF28	Yes			
6	✔		out HEX0[5]	Location	PIN_AG28	Yes			
7	✔		out HEX0[6]	Location	PIN_AH28	Yes			
8	✔		out HEX1[0]	Location	PIN_AJ29	Yes			
9	✔		out HEX1[1]	Location	PIN_AH29	Yes			
10	✔		out HEX1[2]	Location	PIN_AH30	Yes			
11	✔		out HEX1[3]	Location	PIN_AG30	Yes			
12	✔		out HEX1[4]	Location	PIN_AF29	Yes			
13	✔		out HEX1[5]	Location	PIN_AF30	Yes			
14	✔		out HEX1[6]	Location	PIN_AD27	Yes			
15	✔		in en	Location	PIN_AC12	Yes			
16	✔		in reset	Location	PIN_AB12	Yes			
17	✔		out q[0]	Location	PIN_V16	Yes			
18	✔		out q[1]	Location	PIN_W16	Yes			
19	✔		out q[2]	Location	PIN_V17	Yes			
20	✔		out q[3]	Location	PIN_V18	Yes			
21	✔		out q[4]	Location	PIN_W17	Yes			
22	✔		out q[5]	Location	PIN_W19	Yes			
23	✔		out q[6]	Location	PIN_Y19	Yes			
24	✔		out q[7]	Location	PIN_W20	Yes			
25	✔		in clk	Location	PIN_AA14	Yes			
26		<<new>>	<<new>>	<<new>>					

Figure: Pin Assignment

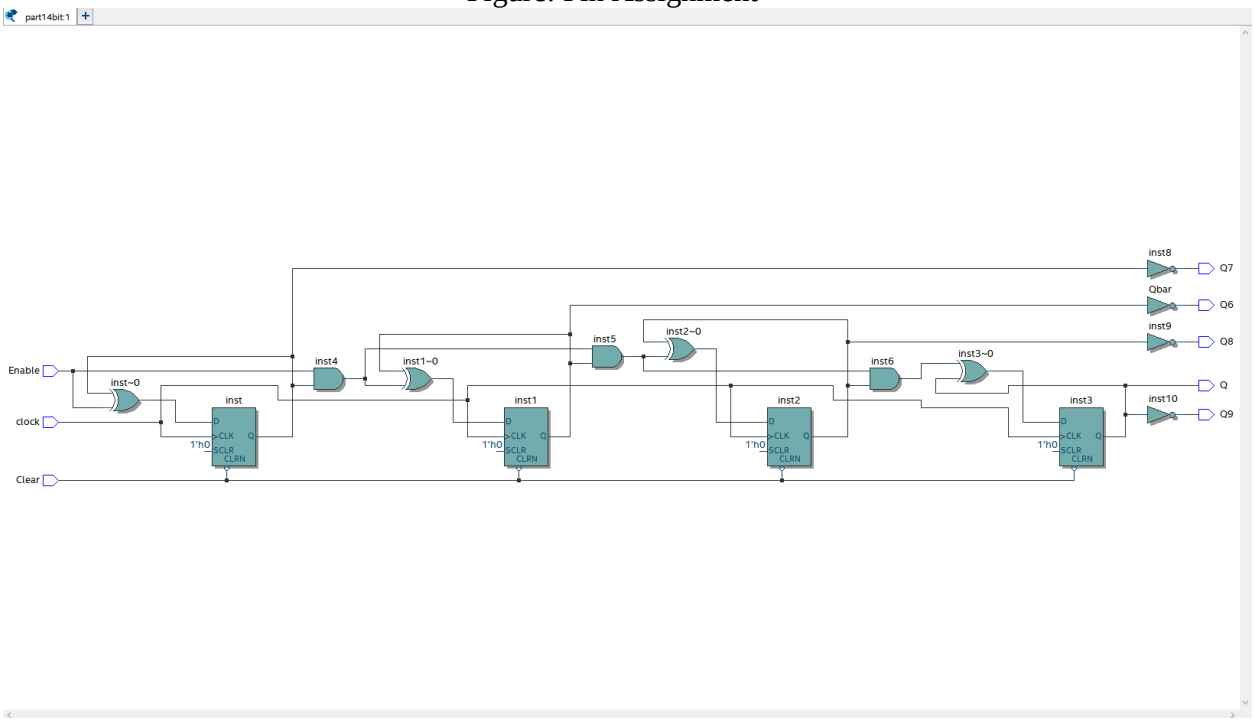


Figure: RTL view of 4 bit counter

Output:

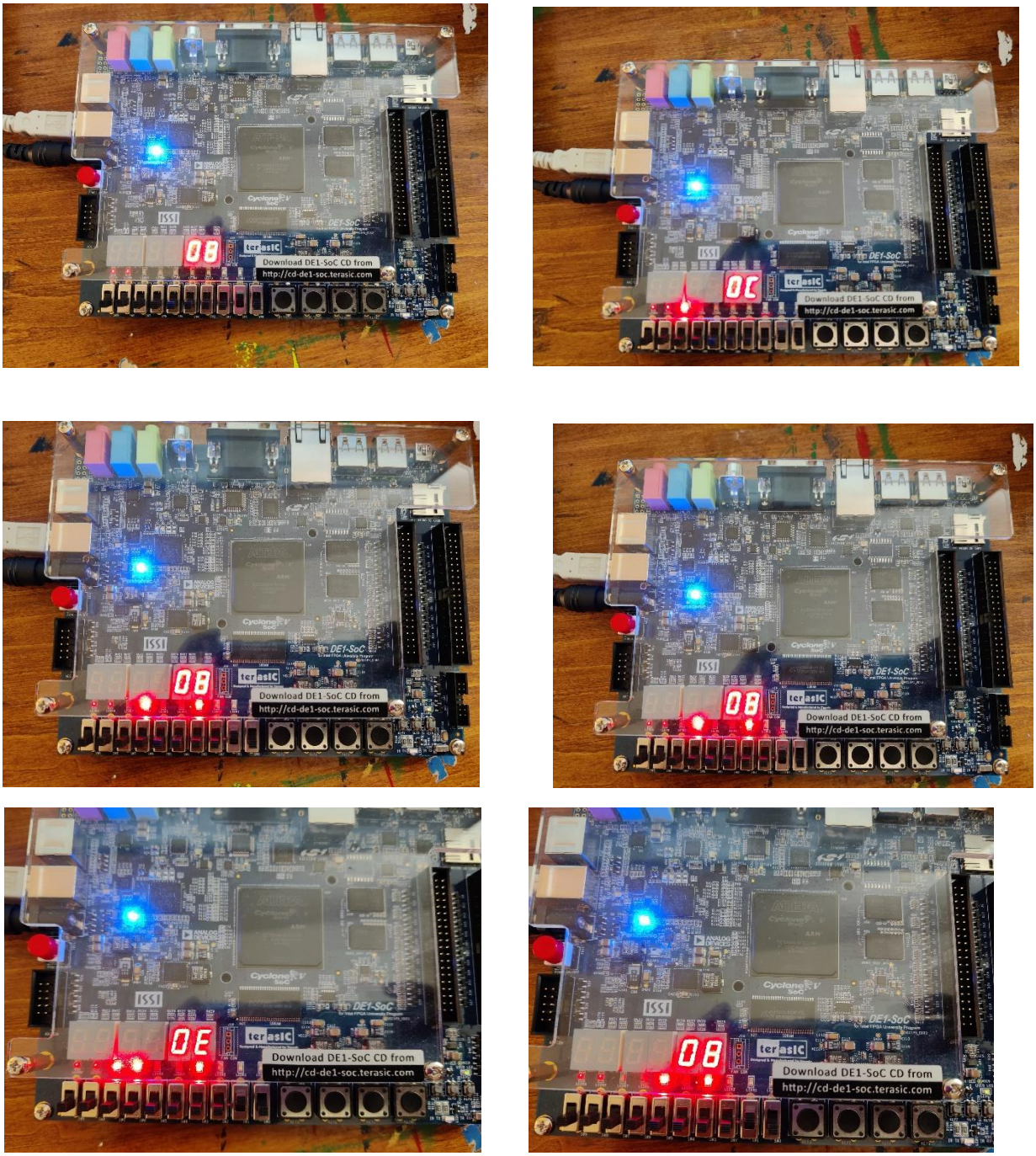
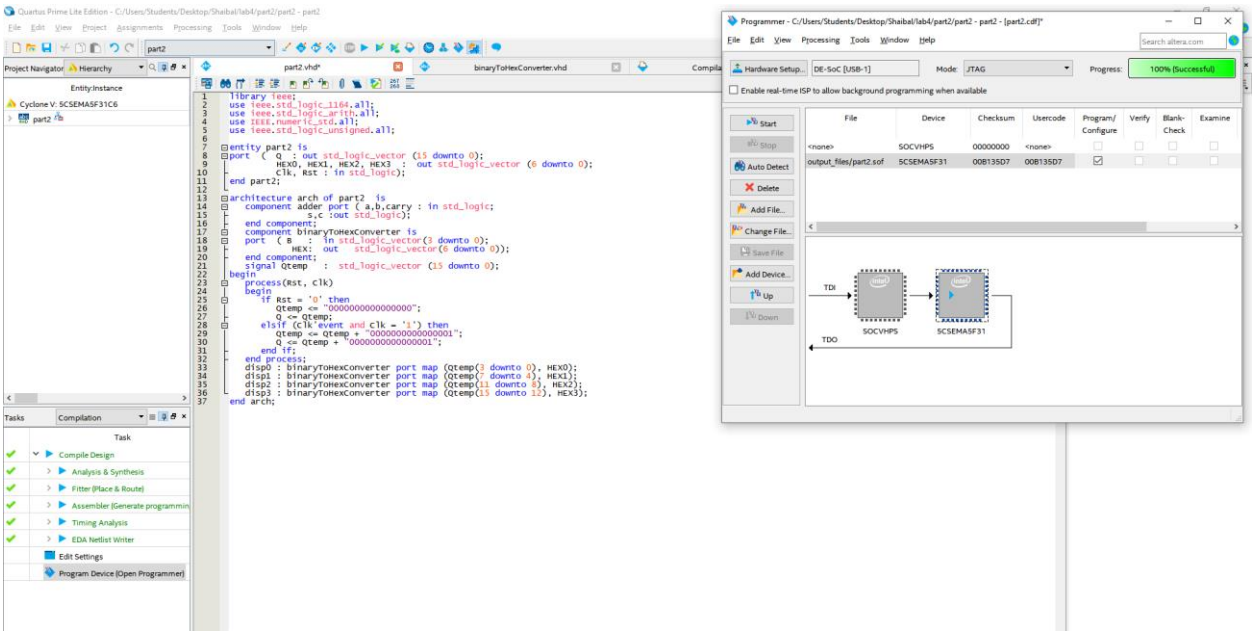


Figure: Output

Part 2: This is the extended version of part1. Here We design a 16-bit counter and display them in 4 7-segment display. The screenshot of outputs and codes are given below:

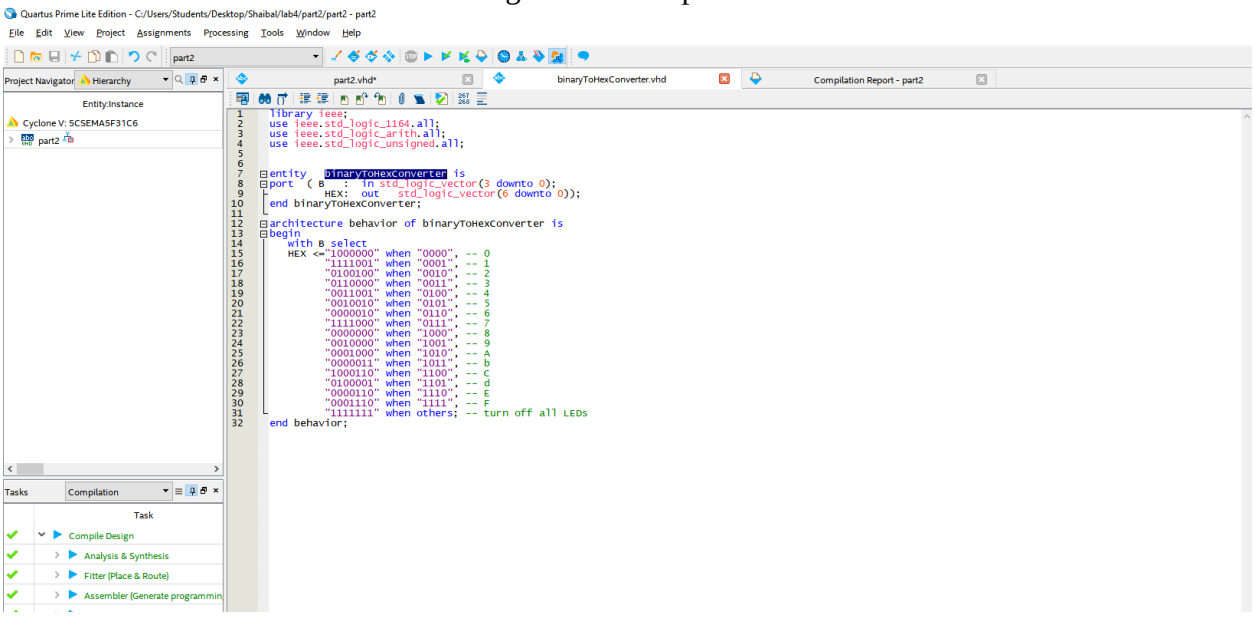


```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use IEEE.numeric_std.all;
use ieee.std_logic_unsigned.all;

entity part2 is
port  (  Q   : out std_logic_vector (15 downto 0);
         HEX0, HEX1, HEX2, HEX3  :  out std_logic_vector (6 downto 0);
         Clk, Rst : in std_logic);
end part2;

architecture arch of part2  is
   component adder port ( a,b,carry : in std_logic;
                          s,C :out std_logic);
   end component;
   component binaryToHexConverter is
   port  ( B   :  in std_logic_vector(3 downto 0);
           HEX:  out   std_logic_vector(6 downto 0));
   end component;
   signal Qtemp  :  std_logic_vector (15 downto 0);
begin
   process(Rst, Clk)
   begin
      if Rst = '0' then
         Qtemp <= "0000000000000000";
         Q <= Qtemp;
      elsif (Clk'event and Clk = '1') then
         Qtemp <= Qtemp + "0000000000000001";
         Q <= Qtemp + "0000000000000001";
      end if;
   end process;
   disp0 : binaryToHexConverter port map (Qtemp(3 downto 0), HEX0);
   disp1 : binaryToHexConverter port map (Qtemp(7 downto 4), HEX1);
   disp2 : binaryToHexConverter port map (Qtemp(11 downto 8), HEX2);
   disp3 : binaryToHexConverter port map (Qtemp(15 downto 12), HEX3);
end arch;
```

Figure: Code of part2



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;


entity   binaryToHexConverter is
port  ( B   :  in std_logic_vector(3 downto 0);
        HEX:  out   std_logic_vector(6 downto 0));
end binaryToHexConverter;

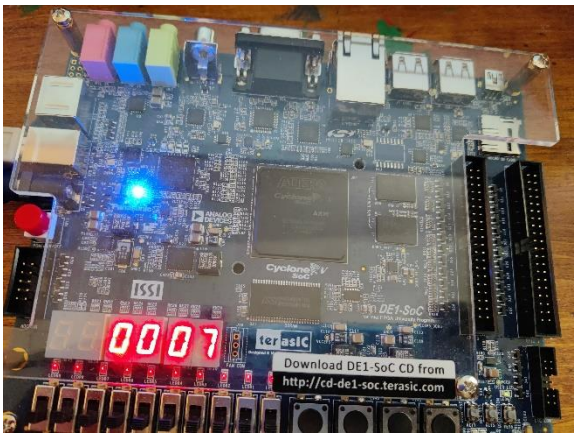
architecture behavior of binaryToHexConverter is
begin
    with B select
    HEX <="1000000" when "0000", -- 0
          "1111001" when "0001", -- 1
          "0100100" when "0010", -- 2
          "0110000" when "0011", -- 3
          "0011001" when "0100", -- 4
          "0010010" when "0101", -- 5
          "0000010" when "0110", -- 6
          "1111000" when "0111", -- 7
          "0000000" when "1000", -- 8
          "0010000" when "1001", -- 9
          "0001000" when "1010", -- A
          "0000011" when "1011", -- b
          "1000110" when "1100", -- C
          "0100001" when "1101", -- d
          "0000110" when "1110", -- E
          "0001110" when "1111", -- F
          "1111111" when others; -- turn off all LEDs
end behavior;
```

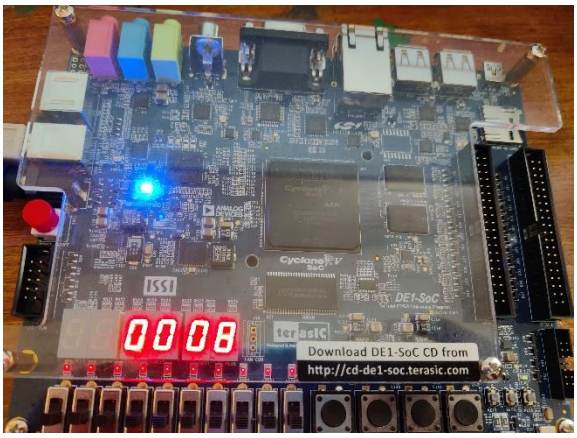
Figure: Code of part2

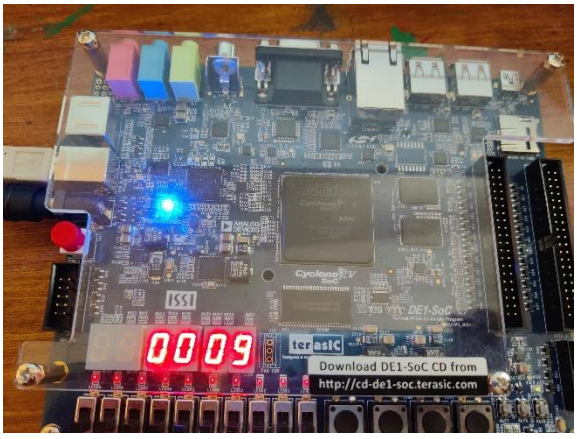
	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔		Rst	Location	PIN_AA15	Yes			
2	✔		HEX0[0]	Location	PIN_AE26	Yes			
3	✔		HEX0[1]	Location	PIN_AE27	Yes			
4	✔		HEX0[2]	Location	PIN_AE28	Yes			
5	✔		HEX0[3]	Location	PIN_AG27	Yes			
6	✔		HEX0[4]	Location	PIN_AF28	Yes			
7	✔		HEX0[5]	Location	PIN_AG28	Yes			
8	✔		HEX0[6]	Location	PIN_AH28	Yes			
9	✔		HEX1[0]	Location	PIN_AJ29	Yes			
10	✔		HEX1[1]	Location	PIN_AH29	Yes			
11	✔		HEX1[2]	Location	PIN_AH30	Yes			
12	✔		HEX1[3]	Location	PIN_AG30	Yes			
13	✔		HEX1[4]	Location	PIN_AF29	Yes			
14	✔		HEX1[5]	Location	PIN_AF30	Yes			
15	✔		HEX1[6]	Location	PIN_AD27	Yes			
16	✔		HEX2[0]	Location	PIN_AB23	Yes			
17	✔		HEX2[1]	Location	PIN_AE29	Yes			
18	✔		HEX2[2]	Location	PIN_AD29	Yes			
19	✔		HEX2[3]	Location	PIN_AC28	Yes			
20	✔		HEX2[4]	Location	PIN_AD30	Yes			
21	✔		HEX2[5]	Location	PIN_AC29	Yes			
22	✔		HEX2[6]	Location	PIN_AC30	Yes			
23	✔		HEX3[0]	Location	PIN_AD26	Yes			
24	✔		HEX3[1]	Location	PIN_AC27	Yes			
25	✔		HEX3[2]	Location	PIN_AD25	Yes			
26	✔		HEX3[3]	Location	PIN_AC25	Yes			
27	✔		HEX3[4]	Location	PIN_AB28	Yes			
28	✔		HEX3[5]	Location	PIN_AB25	Yes			
29	✔		HEX3[6]	Location	PIN_AB22	Yes			
30	✔		Clk	Location	PIN_AA14	Yes			
31		<<new>>	<<new>>	<<new>>					

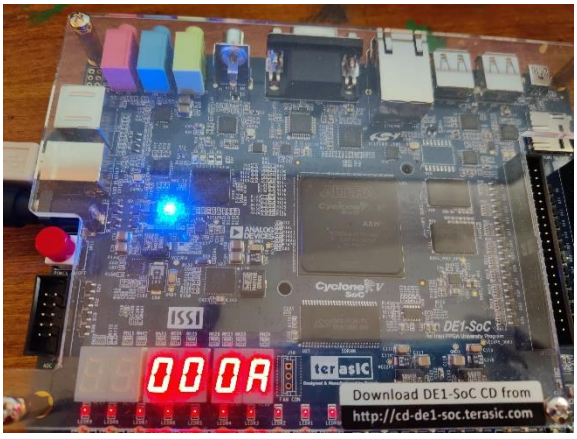
Figure: Pin Assignment

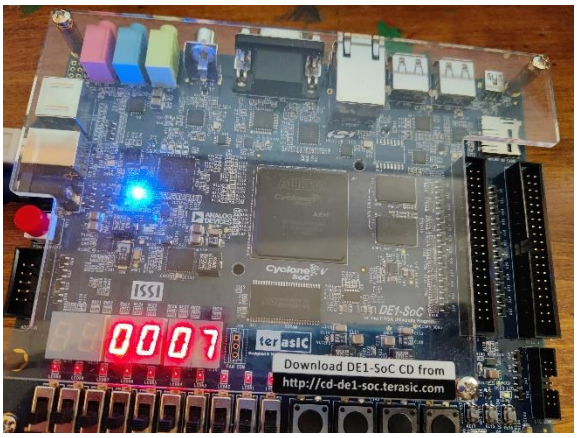
Output:

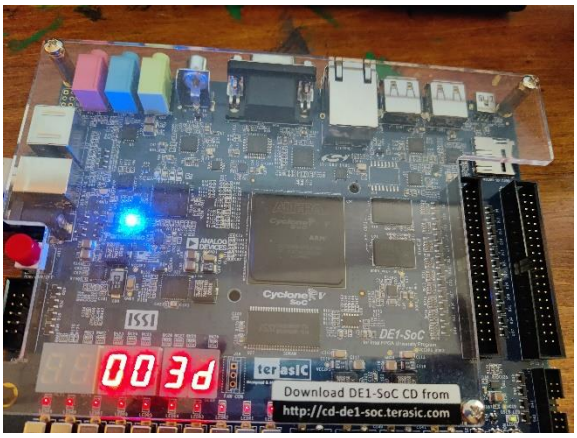












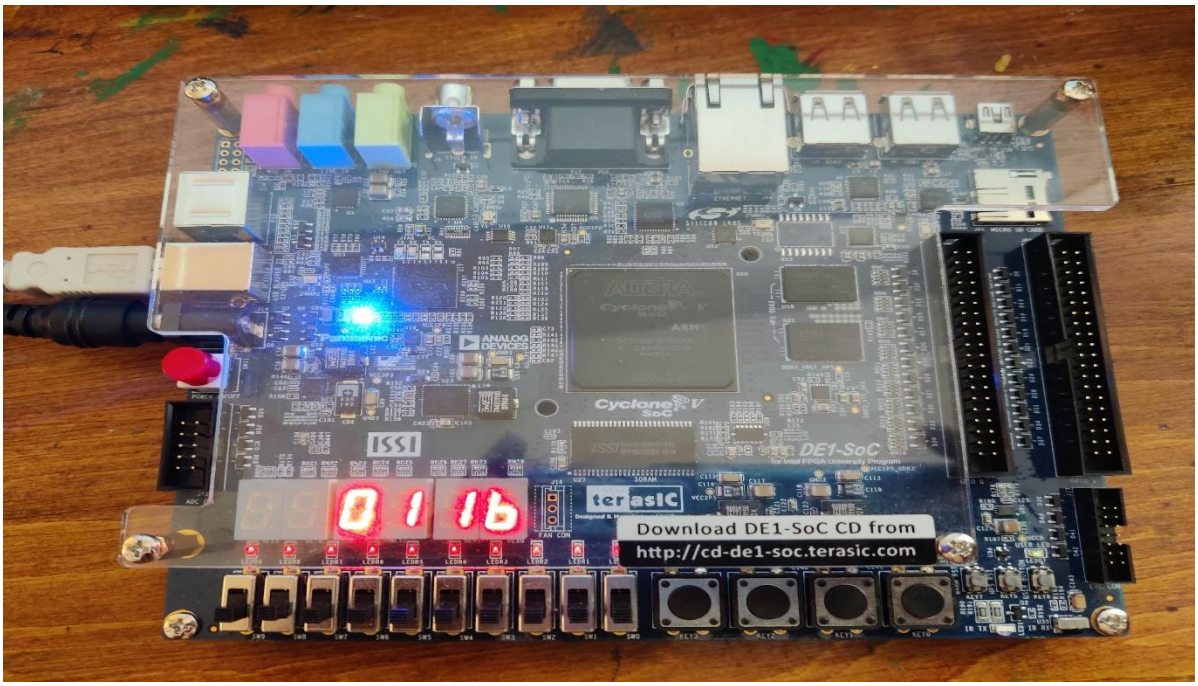


Figure: Output

Part 3: Here, I design a counter o to 9 with a 50 s pause. I got a noise in my DESOC1 board for this problem. The other than Hex0 7 segment display has some noise in this problem, I guess that it is a machine error.The screenshot of outputs and codes are given below:

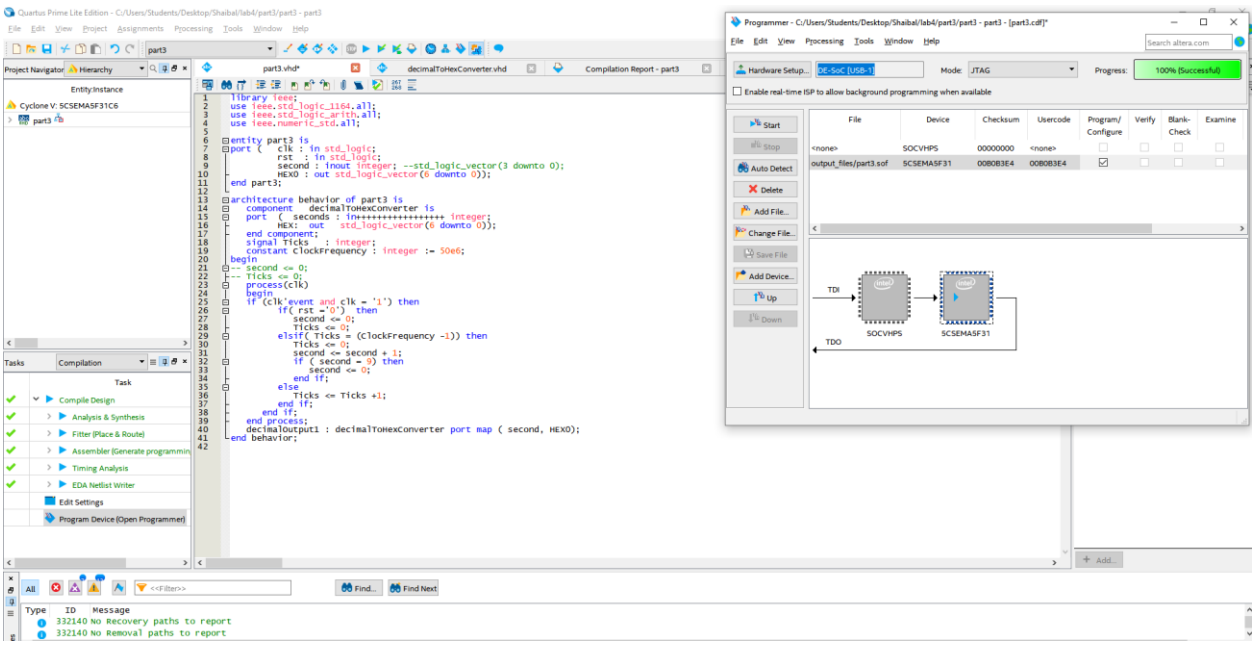


Figure: Code of part3

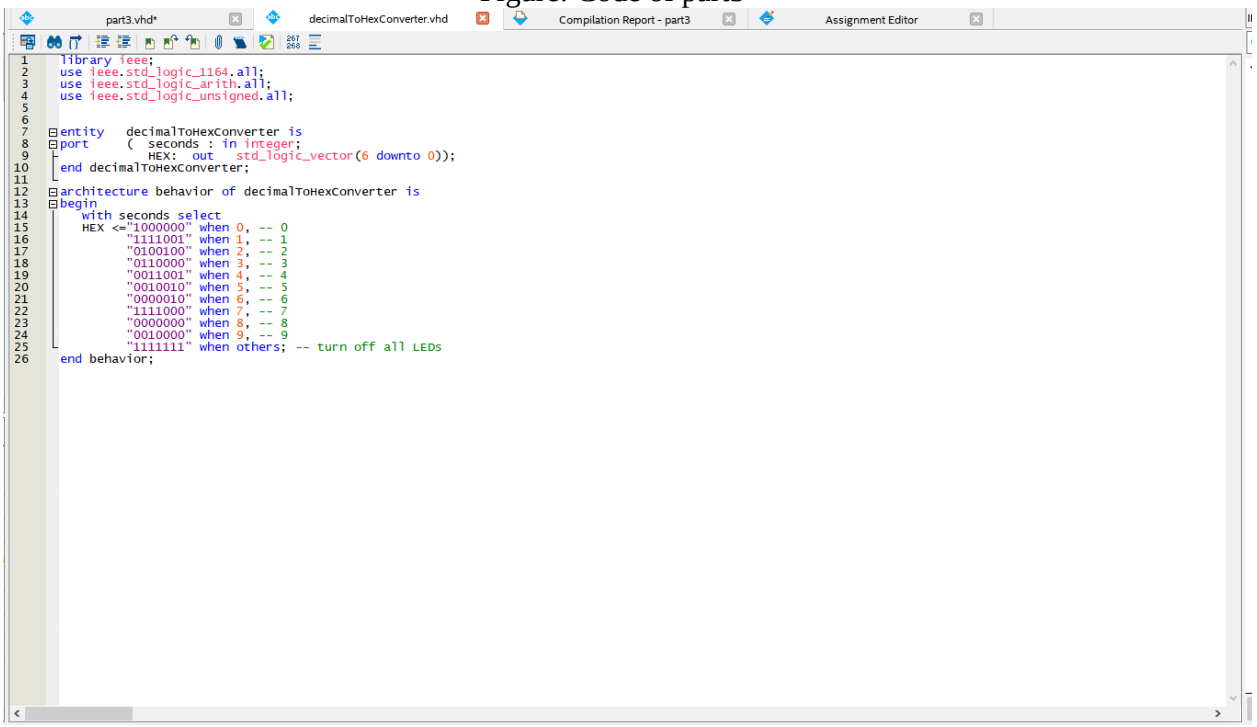


Figure: Code of part3

Output:

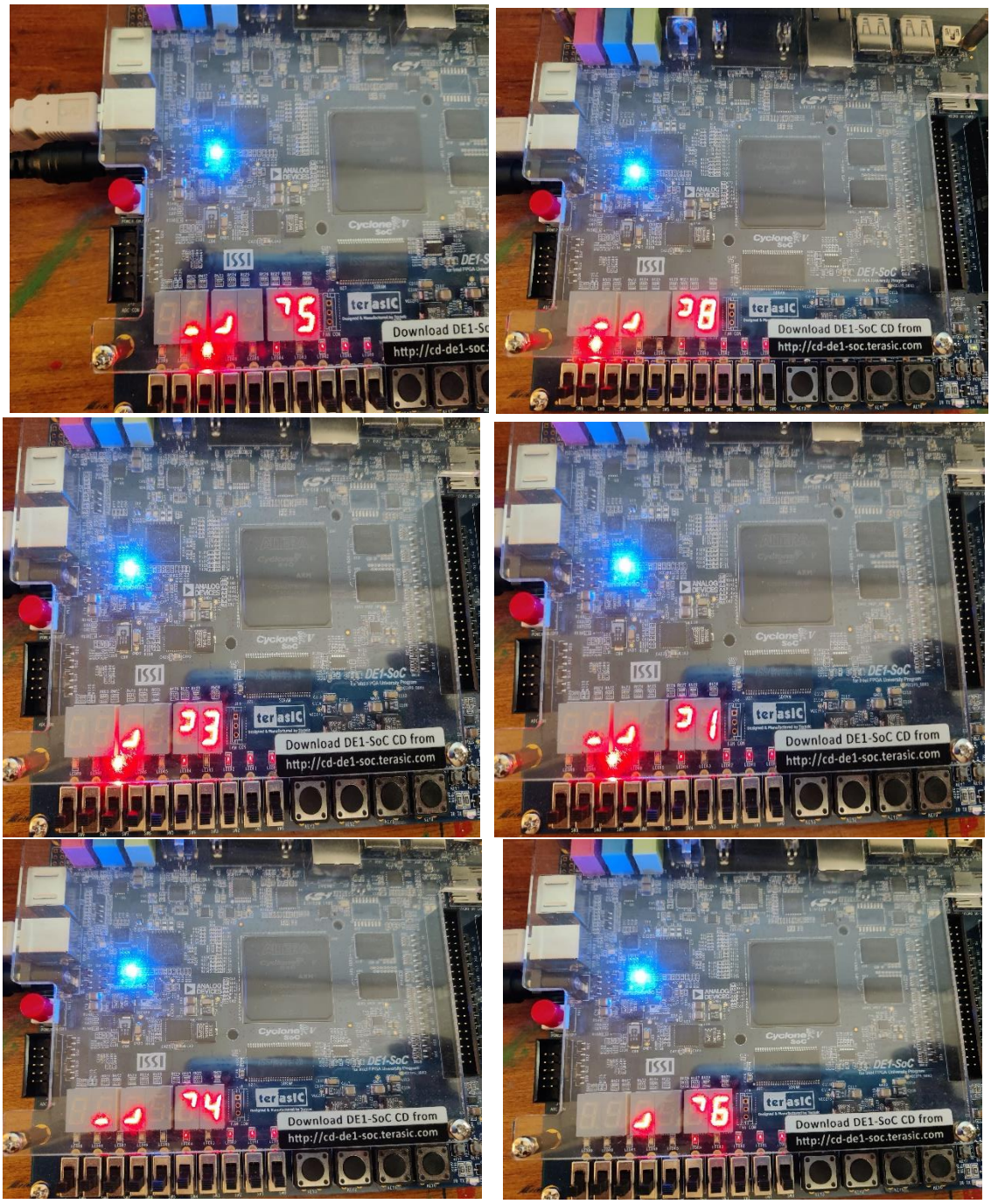
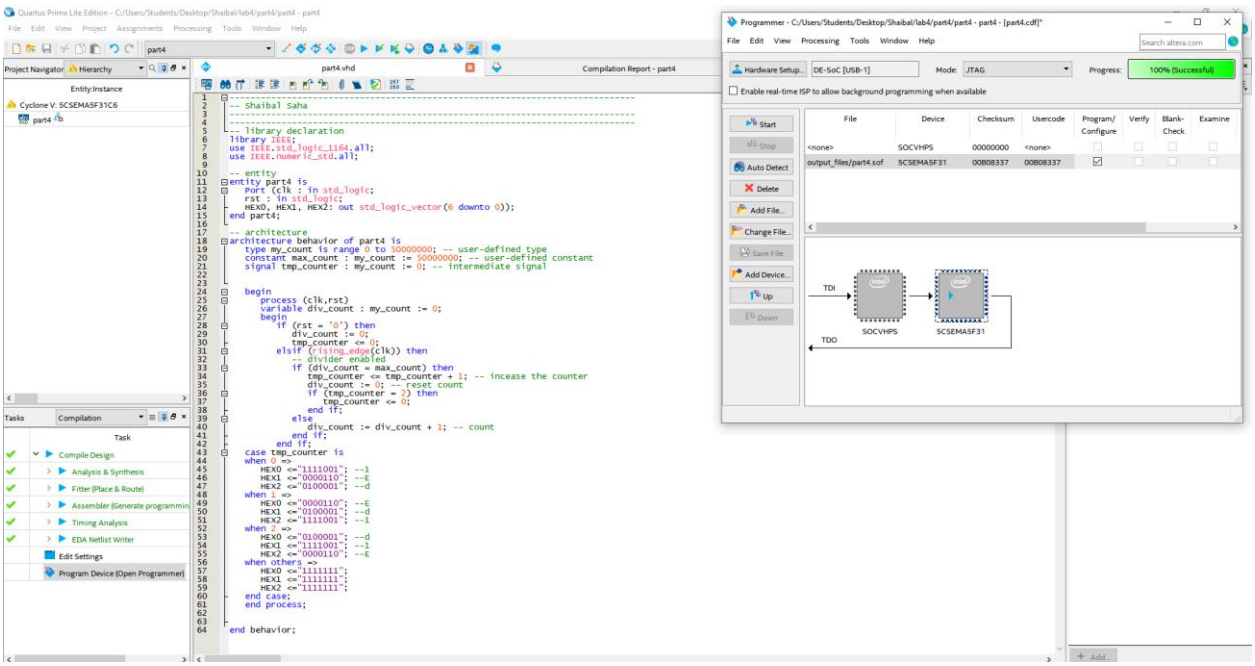


Figure: Output

Part 4: In part4, I need to use the 3 7 segment display and shift right to left the word de1 with a clock 50 seconds delay. The screenshot of outputs and codes are given below:



```
-- Shaibal Saha
-- library declaration
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

-- entity
entity part4 is
    port (clk : in std_logic;
    rst : in std_logic;
    HEX0, HEX1, HEX2: out std_logic_vector(6 downto 0));
end part4;

-- architecture
architecture behavior of part4 is
    type my_count is range 0 to 50000000; -- user-defined type
    constant max_count : my_count := 50000000; -- user-defined constant
    signal tmp_counter : my_count := 0; -- intermediate signal

    begin
        process (clk,rst)
        variable div_count : my_count := 0;
        begin
            if (rst = '0') then
                div_count := 0;
                tmp_counter <= 0;
            elsif (rising_edge(clk)) then
                -- divider enabled
                if (div_count = max_count) then
                    tmp_counter <= tmp_counter + 1; -- increase the counter
                    div_count := 0; -- reset count
                    if (tmp_counter = 2) then
                        tmp_counter <= 0;
                    end if;
                else
                    div_count := div_count + 1; -- count
                end if;
            end if;
    case tmp_counter is
    when 0 =>
        HEX0 <="1111001"; --1
        HEX1 <="0000110"; --E
        HEX2 <="0100001"; --d
    when 1 =>
        HEX0 <="0000110"; --E
        HEX1 <="0100001"; --d
        HEX2 <="1111001"; --1
    when 2 =>
        HEX0 <="0100001"; --d
        HEX1 <="1111001"; --1
        HEX2 <="0000110"; --E
    when others =>
        HEX0 <="1111111";
        HEX1 <="1111111";
        HEX2 <="1111111";
    end case;
    end process;

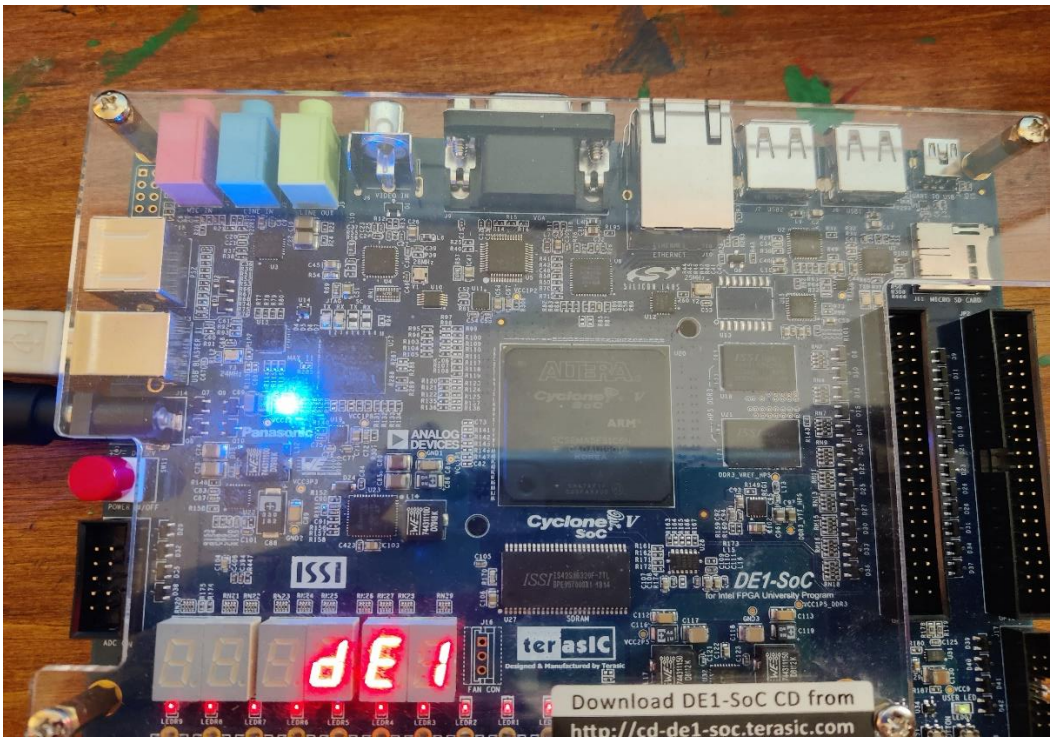
end behavior;
```

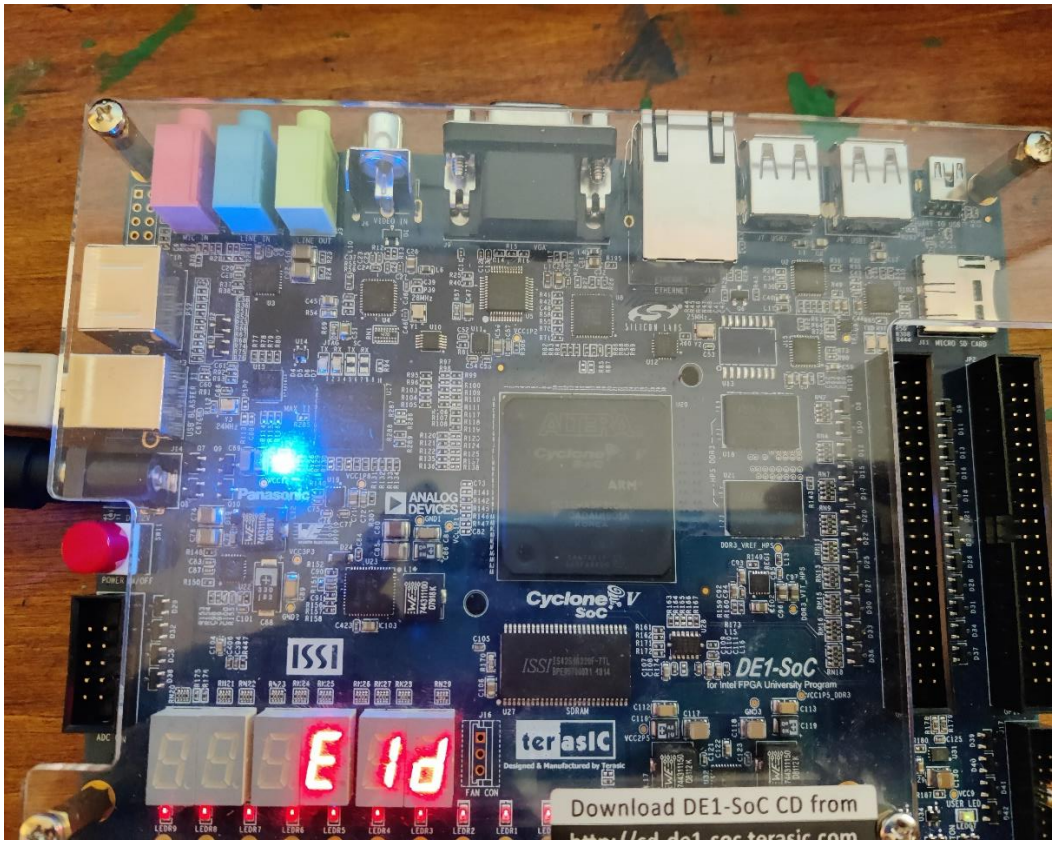
Figure: Code of part4

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔		in rst	Location	PIN_AA14	Yes			
2	✔		out HEX1[1]	Location	PIN_AH29	Yes			
3	✔		out HEX1[2]	Location	PIN_AH30	Yes			
4	✔		out HEX1[3]	Location	PIN_AG30	Yes			
5	✔		out HEX1[4]	Location	PIN_AF29	Yes			
6	✔		out HEX1[5]	Location	PIN_AF30	Yes			
7	✔		out HEX1[6]	Location	PIN_AD27	Yes			
8	✔		out HEX2[0]	Location	PIN_AB23	Yes			
9	✔		out HEX2[1]	Location	PIN_AE29	Yes			
10	✔		out HEX2[2]	Location	PIN_AD29	Yes			
11	✔		out HEX2[3]	Location	PIN_AC28	Yes			
12	✔		out HEX2[4]	Location	PIN_AD30	Yes			
13	✔		out HEX2[5]	Location	PIN_AC29	Yes			
14	✔		out HEX2[6]	Location	PIN_AC30	Yes			
15	✔		out HEX1[0]	Location	PIN_AJ29	Yes			
16	✔		out HEX0[0]	Location	PIN_AE26	Yes			
17	✔		out HEX0[1]	Location	PIN_AE27	Yes			
18	✔		out HEX0[2]	Location	PIN_AE28	Yes			
19	✔		out HEX0[3]	Location	PIN_AG27	Yes			
20	✔		out HEX0[4]	Location	PIN_AF28	Yes			
21	✔		out HEX0[5]	Location	PIN_AG28	Yes			
22	✔		out HEX0[6]	Location	PIN_AH28	Yes			
23	✔		in clk	Location	PIN_AF14	Yes			
24		<<new>>	<<new>>	<<new>>					

Figure: Pin assignment

Output:


dE1
Download DE1-SoC CD from
http://cd-de1-soc.terasic.com


E1d
Download DE1-SoC CD from

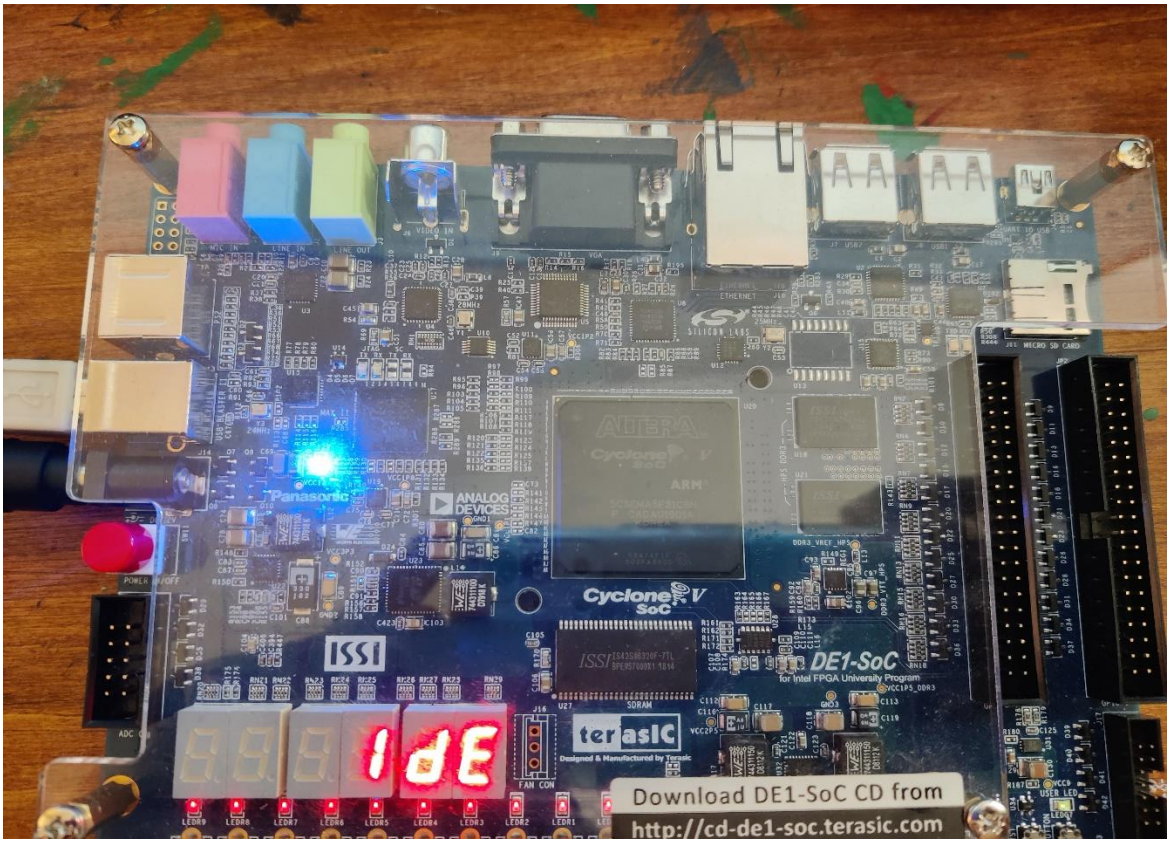


Figure: Output

Part 5: In part 5, We reuse the code from lab 1. We use a counter to shift the letter within those 5 seven segments. The screenshot of outputs and codes are given below:

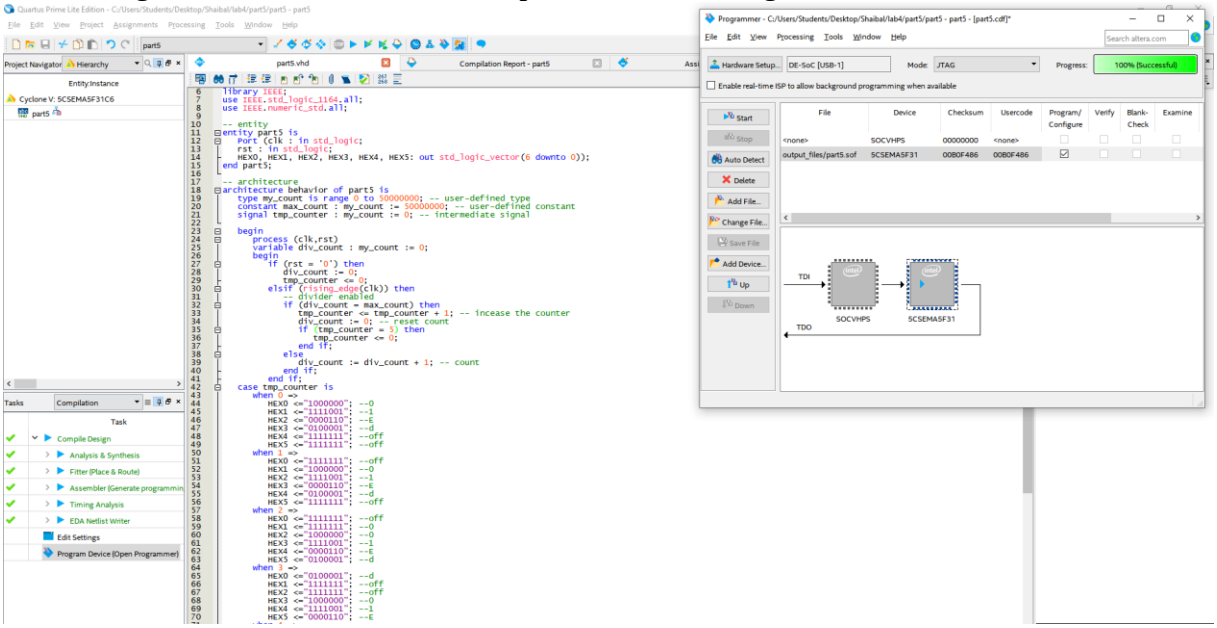


Figure: Code of part5

```
        when 4 =>
            HEX0 <="0000110"; --E
            HEX1 <="0100001"; --d
            HEX2 <="1111111"; --off
            HEX3 <="1111111"; --off
            HEX4 <="1000000"; --0
            HEX5 <="1111001"; --1
        when 5 =>
            HEX0 <="1111001"; --1
            HEX1 <="0000110"; --E
            HEX2 <="0100001"; --d
            HEX3 <="1111111"; --off
            HEX4 <="1111111"; --off
            HEX5 <="1000000"; --0
        when others =>
            HEX0 <="1111111"; --off
            HEX1 <="1111111"; --off
            HEX2 <="1111111"; --off
            HEX3 <="1111111"; --off
            HEX4 <="1111111"; --off
            HEX5 <="1111111"; --off
    end case;
  end process;
end behavior;
```

Figure: Code of part5

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✓		in rst	Location	PIN_AA14	Yes			
2	✓		out HEX1[1]	Location	PIN_AH29	Yes			
3	✓		out HEX1[2]	Location	PIN_AH30	Yes			
4	✓		out HEX1[3]	Location	PIN_AG30	Yes			
5	✓		out HEX1[4]	Location	PIN_AF29	Yes			
6	✓		out HEX1[5]	Location	PIN_AF30	Yes			
7	✓		out HEX1[6]	Location	PIN_AD27	Yes			
8	✓		out HEX2[0]	Location	PIN_AB23	Yes			
9	✓		out HEX2[1]	Location	PIN_AE29	Yes			
10	✓		out HEX2[2]	Location	PIN_AD29	Yes			
11	✓		out HEX2[3]	Location	PIN_AC28	Yes			
12	✓		out HEX2[4]	Location	PIN_AD30	Yes			
13	✓		out HEX2[5]	Location	PIN_AC29	Yes			
14	✓		out HEX2[6]	Location	PIN_AC30	Yes			
15	✓		out HEX1[0]	Location	PIN_AJ29	Yes			
16	✓		out HEX0[0]	Location	PIN_AE26	Yes			
17	✓		out HEX0[1]	Location	PIN_AE27	Yes			
18	✓		out HEX0[2]	Location	PIN_AE28	Yes			
19	✓		out HEX0[3]	Location	PIN_AG27	Yes			
20	✓		out HEX0[4]	Location	PIN_AF28	Yes			
21	✓		out HEX0[5]	Location	PIN_AG28	Yes			
22	✓		out HEX0[6]	Location	PIN_AH28	Yes			
23	✓		out HEX3[1]	Location	PIN_AC27	Yes			
24	✓		out HEX3[2]	Location	PIN_AD25	Yes			
25	✓		out HEX3[3]	Location	PIN_AC25	Yes			
26	✓		out HEX3[4]	Location	PIN_AB28	Yes			
27	✓		out HEX3[5]	Location	PIN_AB25	Yes			
28	✓		out HEX3[6]	Location	PIN_AB22	Yes			
29	✓		out HEX4[0]	Location	PIN_AA24	Yes			
30	✓		out HEX4[1]	Location	PIN_Y23	Yes			
31	✓		out HEX4[2]	Location	PIN_Y24	Yes			
32	✓		out HEX4[3]	Location	PIN_W22	Yes			
33	✓		out HEX4[4]	Location	PIN_W24	Yes			
34	✓		out HEX4[5]	Location	PIN_V23	Yes			
35	✓		out HEX4[6]	Location	PIN_W25	Yes			
36	✓		out HEX5[0]	Location	PIN_V25	Yes			
37	✓		out HEX5[1]	Location	PIN_AA28	Yes			
38	✓		out HEX5[2]	Location	PIN_Y27	Yes			
39	✓		out HEX5[3]	Location	PIN_AB27	Yes			
40	✓		out HEX5[4]	Location	PIN_AB26	Yes			
41	✓		out HEX5[5]	Location	PIN_AA26	Yes			
42	✓		out HEX5[6]	Location	PIN_AA25	Yes			
43	✓		out HEX3[0]	Location	PIN_AD26	Yes			
44	✓		in clk	Location	PIN_AF14	Yes			
45		<<new>>	<<new>>	<<new>>					

Figure: Pin assignment

Output:

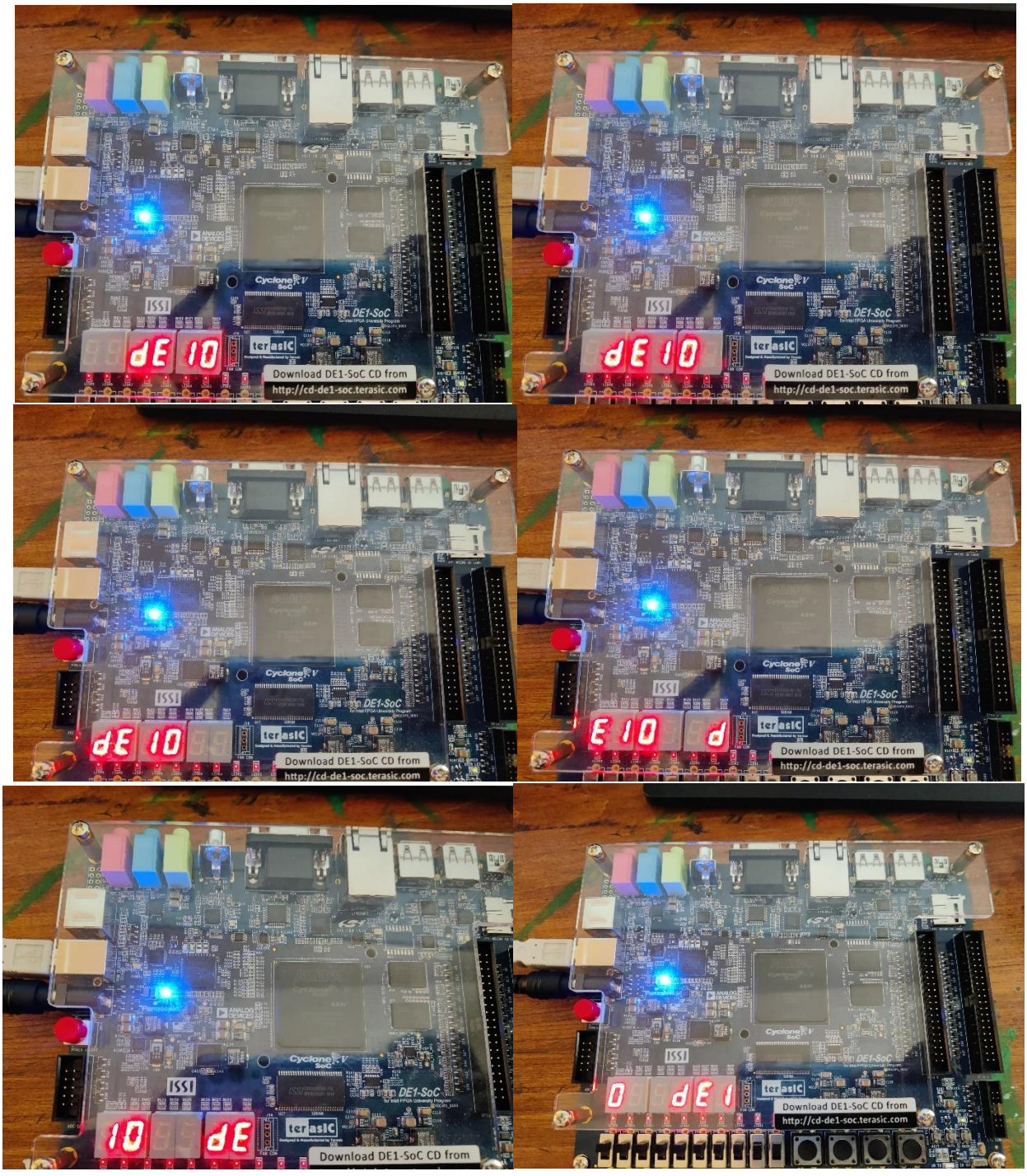


Figure: Output

Note: All the source codes are inside lab4 folder.